

Smart City Complaint Management System

A serverless civic complaint management platform built on AWS that allows citizens to submit complaints using images, automatically analyzes uploaded images using Amazon Rekognition, stores complaint information in DynamoDB, generates audio responses using Amazon Polly, and sends notifications through SNS.

The application is delivered through Amazon CloudFront with authentication supported by Amazon Cognito.

1. Project Overview

The Smart City Complaint Management System is a cloud-based application designed to digitize the process of reporting and managing civic issues such as garbage, potholes, streetlights, traffic-related problems and other public infrastructure issues.

Instead of relying on manual complaint registration, the system allows citizens to submit an image and location through a web application. AWS services process the complaint automatically and provide structured information to the backend.

Project Type: Serverless AWS application

Region: ap-south-1 (Mumbai)

Backend: Python + AWS Lambda + API Gateway

Database: Amazon DynamoDB

AI: Amazon Rekognition

Authentication: Amazon Cognito

2. Problem Statement

Traditional civic complaint systems may require manual registration, limited information and separate communication channels. This can make it difficult to classify complaints, track their status and notify relevant users efficiently.

This project demonstrates how serverless cloud services and AI can be combined to create a centralized complaint-management workflow.

3. Project Objectives

Citizen Reporting

Allow citizens to submit civic complaints with location and image information.

AI Analysis

Analyze uploaded images using Amazon Rekognition.

Centralized Data

Store complaint records in Amazon DynamoDB.

Audio Response

Generate spoken complaint information using Amazon Polly.

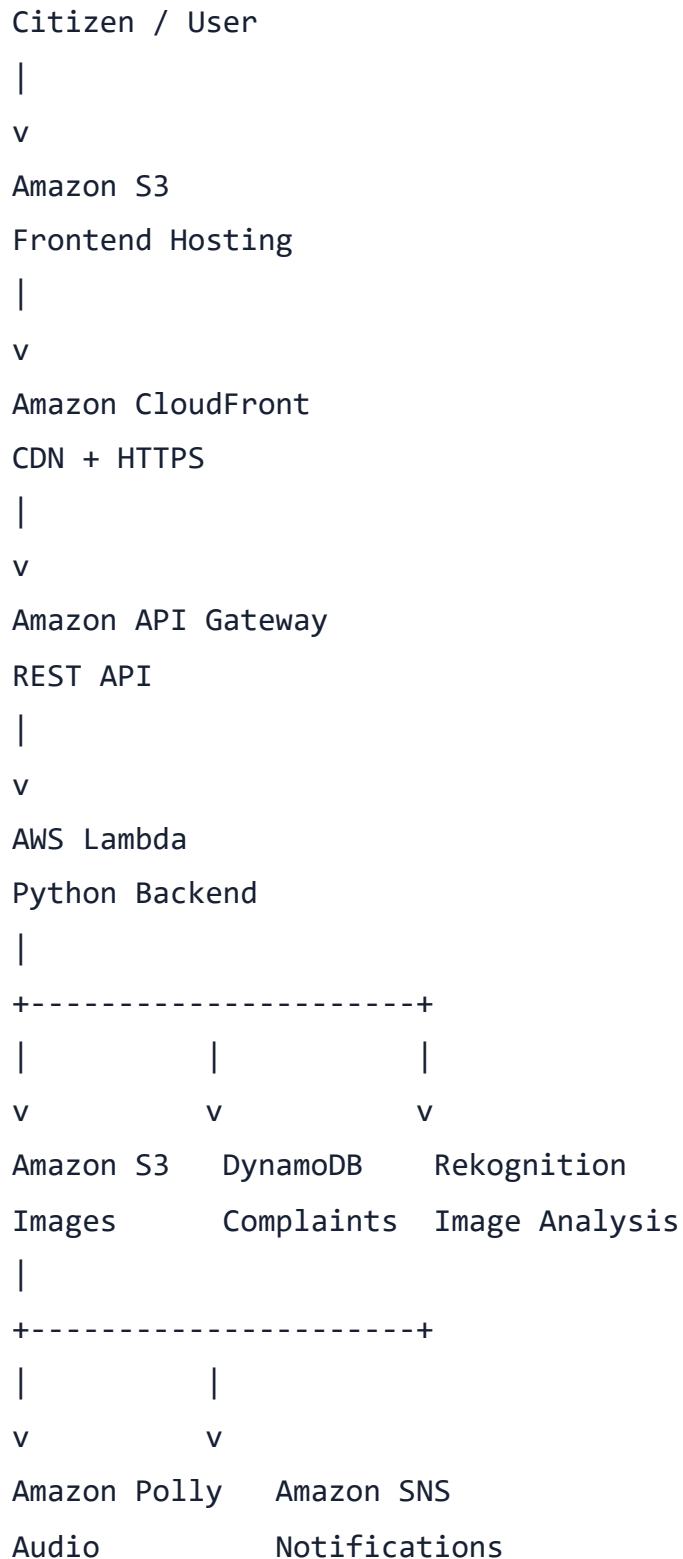
Notifications

Send complaint notifications using Amazon SNS.

Secure Access

Use Amazon Cognito to support authenticated application access.

4. System Architecture



```
Administrator
|
v
Amazon Cognito
Authentication
|
v
Authenticated Application
|
v
API Gateway
|
v
Lambda → DynamoDB
```

The architecture is serverless, meaning the application does not require a traditional always-running backend server.

The architecture diagram is also available in the project's [GitHub architecture folder](#).

5. AWS Services Used

Amazon S3

Frontend hosting and storage of complaint images and generated audio files.

Amazon CloudFront

Global content delivery, HTTPS access and public delivery of the web application.

Amazon Cognito

Authentication layer for authenticated application/admin access.

Amazon API Gateway

REST API entry point between the frontend and Lambda backend.

AWS Lambda

Serverless Python backend responsible for complaint processing.

Amazon DynamoDB

NoSQL database storing complaint information.

Amazon Rekognition

Image label detection used for complaint analysis.

Amazon Polly

Converts complaint information into speech and generates MP3 audio.

Amazon SNS

Sends complaint notifications.

6. End-to-End Workflow

1 Citizen opens the portal

The frontend is delivered through Amazon CloudFront.

2 Complaint submission

The citizen selects a location and uploads an image.

3 API Gateway receives the request

The REST API forwards the request to AWS Lambda.

4 Lambda creates a complaint ID

A unique ID such as `CMP-XXXXXXX` is generated.

5 Image stored in S3

The Base64 image is decoded and uploaded to Amazon S3.

6 Rekognition analyzes the image

Amazon Rekognition detects labels from the uploaded image.

7 Issue classification

Detected labels are matched against complaint-category keywords.

8 Complaint stored

Complaint information is stored in DynamoDB.

9 Audio generated

Amazon Polly converts complaint information into an MP3 file.

10 Notification sent

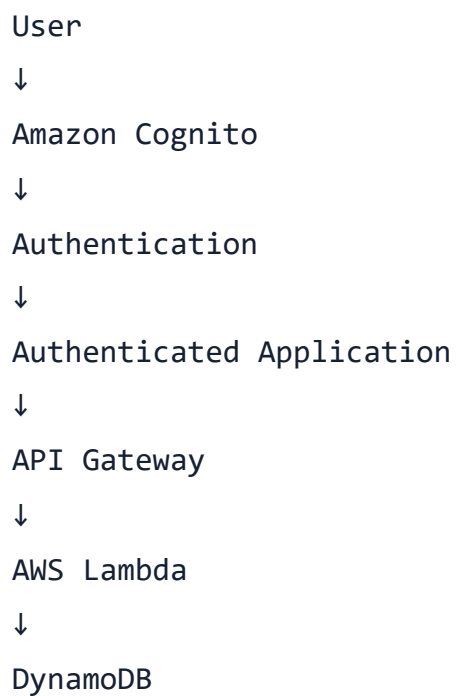
Amazon SNS publishes a complaint notification.

11 Result returned

The API returns complaint ID, issue, Rekognition labels, status and audio information.

7. Amazon Cognito Authentication

Amazon Cognito is used as the authentication layer for the application. It provides an identity mechanism for authenticated users and administrative access rather than exposing administrative functionality as an unrestricted public interface.



Cognito can also be extended with stronger role-based authorization, such as separate citizen and administrator permissions.

8. Backend Implementation

The backend is implemented using Python and AWS Lambda. The Lambda function acts as the central processing layer for the application.

Lambda Responsibilities

- Receive API requests
- Validate complaint data
- Generate complaint IDs
- Decode Base64 images
- Upload images to S3
- Call Amazon Rekognition
- Classify the complaint
- Store records in DynamoDB
- Generate Polly audio
- Store audio in S3
- Publish SNS notifications
- Return API responses

Backend source code is available in the repository's [backend folder](#).

9. Amazon Rekognition Integration

Amazon Rekognition is used to analyze uploaded complaint images. The service detects objects and concepts represented by labels.

```
rekognition.detect_labels(  
    Image={"Bytes": image_bytes},  
    MaxLabels=20,  
    MinConfidence=40  
  
)
```

Issue Classification

Complaint Category	Example Matching Keywords
Garbage	garbage, waste, trash, litter, rubbish, dump, debris
Pothole	pothole, road damage, damaged road, hole, puddle, manhole, tar, street
Streetlight	streetlight, street lamp, lamp, light
Traffic	traffic, car, vehicle, automobile, bus, truck, motorcycle
Unknown	Used when no configured keyword matches the detected labels.

Processing Logic

Uploaded Image



Amazon Rekognition



Detected Labels



Keyword Matching



Complaint Category



DynamoDB

10. Amazon DynamoDB

Complaint records are stored in the DynamoDB table `Complaints` .

Field	Description
complaintId	Unique complaint identifier.
location	Location selected by the citizen.
issue	Classified complaint category.
rekognitionLabels	Labels detected by Amazon Rekognition.
imageName	Original uploaded image filename.
s3Key	Location of the complaint image in S3.
status	Pending, In Progress or Resolved.
createdAt	Complaint creation timestamp.

Complaint Status

- `Pending`
- `In Progress`
- `Resolved`

11. Amazon S3 Storage

Amazon S3 is used for both frontend hosting/storage and application-generated files.

Complaint Images

Bucket: `yaman-complaint-images`

Image key format:

`complaints/{complaintId}-
{imageName}`

Complaint Audio

Generated using Amazon Polly.

Audio key format: `complaint-`

`audio/{complaintId}.mp3`

Temporary access to generated audio can be provided using a presigned S3 URL.

12. Amazon Polly

Amazon Polly converts generated complaint information into speech. The backend generates an MP3 file and stores it in Amazon S3.

```
polly.synthesize_speech(  
    Text=complaint_text,  
    VoiceId="Joanna",  
    OutputFormat="mp3",  
    Engine="standard"  
  
)
```

Complaint Information

↓

Amazon Polly

↓

MP3 Audio

↓

Amazon S3

↓

Presigned Audio URL

13. Amazon SNS Notifications

Amazon SNS is used to publish complaint notifications after successful complaint processing.

The notification can include information such as:

- Complaint ID
- Location
- Detected issue
- Complaint status
- Audio URL

SNS topic used by the application: `SmartCityComplaintAlerts`

14. API Documentation

API Gateway Base URL

<https://7rbz9pjaw8.execute-api.ap-south-1.amazonaws.com>

The exact deployed route or stage should be appended according to the API Gateway deployment configuration.

POST — Create Complaint

Creates a new complaint and starts the complete image-analysis workflow.

```
{

  "location": "Kondhwa",
  "imageName": "garbage1.jpg",
  "image": ""
}
```

Successful Response

```
{

  "message": "Complaint uploaded successfully",
  "complaintId": "CMP-XXXXXXX",
  "location": "Kondhwa",
  "imageName": "garbage1.jpg",
  "s3Key": "complaints/CMP-XXXXXXX-garbage1.jpg",
  "issue": "Garbage",
  "rekognitionLabels": [
    "Garbage",
    "Waste"
  ],
  "status": "Pending",
  "createdAt": "2026-09-10T00:00:00+00:00",
}
```

```
"unresolvedComplaints": 1,  
"audioKey": "complaint-audio/CMP-XXXXXXXXX.mp3",  
"audioUrl": ""  
}
```

GET — Retrieve Complaints

Retrieves complaint records from DynamoDB. The backend scans the complaint table, applies pagination handling and returns the newest records first.

PATCH — Update Complaint Status

Updates the status of an existing complaint.

```
{  
  
"complaintId": "CMP-XXXXXXXX",  
"status": "Resolved"  
}
```

Allowed status values:

- Pending
- In Progress
- Resolved

OPTIONS — CORS Preflight

The API supports OPTIONS requests for browser CORS preflight handling.

CORS Configuration

```
Access-Control-Allow-Origin: *
```

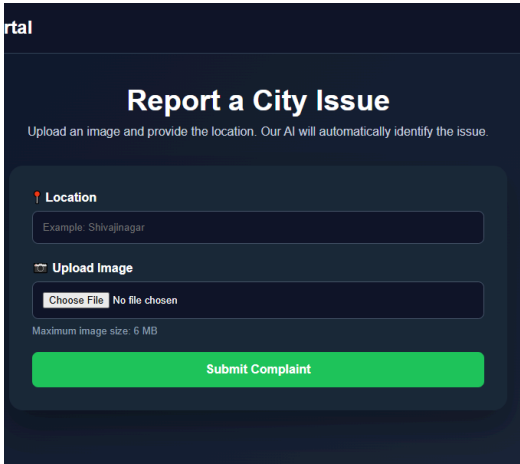
```
Access-Control-Allow-Headers: Content-Type,Authorization
```

```
Access-Control-Allow-Methods: GET,POST,PATCH,OPTIONS
```

For production deployment, CORS should be restricted to the trusted frontend domain rather than using a wildcard origin.

15. Project Evidence & Screenshots

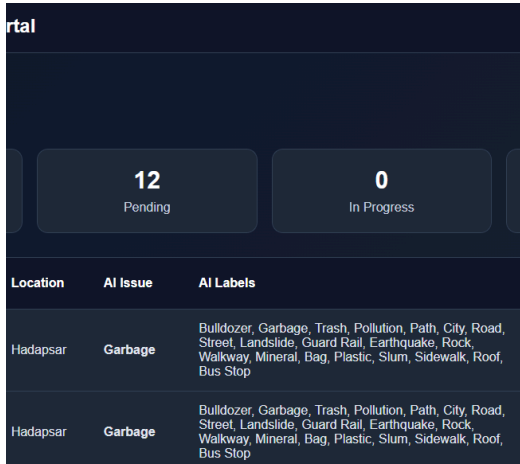
The project repository contains screenshots demonstrating the working application. These screenshots provide visual evidence of the frontend, administrator portal and notification workflow.



Citizen Portal

Main citizen-facing complaint submission interface.

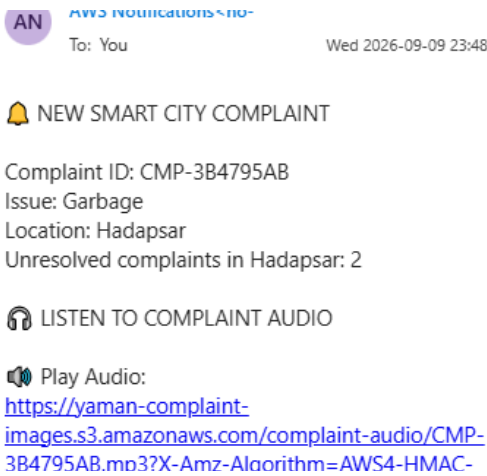
[View on GitHub →](#)



Admin Portal

Administrative interface used for viewing and managing complaints.

[View on GitHub →](#)



SNS Email Notification

Example notification generated through Amazon SNS.

[View on GitHub →](#)

16. Security Considerations

Authentication

Amazon Cognito provides authentication for protected application access.

IAM

AWS IAM permissions should follow the principle of least privilege.

Presigned URLs

Temporary presigned URLs can be used for controlled access to stored files.

S3 Protection

S3 objects should not be exposed publicly unless there is a specific application requirement.

CORS

Production deployments should restrict CORS to the application's trusted domain.

Credentials

AWS access keys, passwords, secrets and credentials must never be committed to GitHub.

17. Future Enhancements

- Expand Cognito role management for citizens, officers and administrators.
- Implement stronger role-based API authorization.
- Add map-based visualization of complaint locations.
- Add automatic escalation for unresolved complaints.
- Build analytics dashboards for complaint trends.
- Improve AI-based complaint classification.
- Add resolution-time analytics.
- Add geographic intelligence for identifying complaint hotspots.
- Introduce automated operational monitoring and alerting.

18. Project Links

Live Application

<https://dp97potsl28ja.cloudfront.net/>

GitHub Repository

<https://github.com/Quazi-Yaman/smart-city-complaint-management>

Architecture

[GitHub Architecture Folder](#)

Backend Source Code

[GitHub Backend Folder](#)

Frontend Source Code

[GitHub Frontend Folder](#)

Screenshots

[GitHub Screenshots Folder](#)

API Gateway

<https://7rbz9pjaw8.execute-api.ap-south-1.amazonaws.com>

API Gateway API ID: `7rbz9pjaw8`

AWS Region: `ap-south-1`

19. Conclusion

The Smart City Complaint Management System demonstrates how AWS serverless services can be combined with artificial intelligence to create a practical cloud-based civic application.

The system integrates Amazon S3, CloudFront, Cognito, API Gateway, Lambda, DynamoDB, Rekognition, Polly and SNS into a complete end-to-end workflow.

The project demonstrates practical experience in serverless architecture, REST API development, NoSQL data storage, image processing, AI integration, authentication, cloud storage, notifications and frontend deployment.

Key takeaway: The project is not simply a static website. It demonstrates an end-to-end AWS serverless architecture where a user action triggers multiple managed cloud services to process, store, analyze and communicate complaint information.

Smart City Complaint Management System

AWS Serverless • AI • Cloud Computing • Python • REST API

Built and documented by Yamania Quazi